

Feuille de travaux dirigés n° 1

Du parallélisme à la concurrence

Exercice 1 (La loi d’Amdahl).

Une proportion $p = 70\%$ d’un programme séquentiel peut être parallélisée. On dispose d’une machine avec $n = 10$ cœurs.

- 1a.** Quelle est l’accélération (speedup) attendue ?
- 1b.** On trouve dans le commerce une machine avec $n = 20$ cœurs. Quelle est l’accélération attendue ?
- 1c.** Quelle est l’accélération maximale théorique lorsque $n \rightarrow \infty$?

Un ingénieur pense qu’il existe un moyen d’optimiser la partie séquentielle pour la rendre deux fois plus rapide.

- 1d.** Quelle proportion du programme est parallélisable dans la version optimisée ?
- 1e.** Quelle accélération obtient-on alors, dans la parallélisation avec $n = 10$ cœurs ?
- 1f.** À coût égal, vaut-il mieux choisir d’acheter la machine avec $n = 20$ cœurs ou payer l’ingénieur pour optimiser le code ?

Exercice 2 (Parallélisation).

Le but de cet exercice est d’écrire une fonction `double[][] product(double[][] M1, double[][] M2)` qui calcule le produit de deux matrices carrées M_1 et M_2 de taille $n \times n$. On rappelle la définition de la multiplication des matrices. Si $M_3 = M_1 \times M_2$, alors pour tout $i, j \in \{0, \dots, n-1\}$,

$$M_3[i][j] = \sum_{k=0}^{n-1} M_1[i][k] \times M_2[k][j].$$

- 2a.** Implémentez la version séquentielle de `product`.
- 2b.** Proposez une parallélisation où chaque cellule $M_3[i][j]$ est calculée par un thread distinct (jusqu’à n^2 threads).
- 2c.** Proposez une version qui crée au plus T threads (T est une variable globale), chacun calculant un lot de cellules. Précisez votre stratégie de répartition.
- 2d.** Analysez la complexité en temps des versions et justifiez l’intérêt de limiter le nombre de threads à une borne indépendante de n .

Exercice 3 (Asynchronie).

On se donne le programme multi-threads suivant :

```
Thread t = new Thread(() -> {  
    System.out.print("a");  
    new Thread(() -> { System.out.print("b"); }).start();  
    System.out.print("c");  
});  
  
System.out.print("d");  
t.start();  
System.out.print("e");  
t.join();  
System.out.print("f");
```

- 3a.** Décrivez la relation “happens-before” du programme.
- 3b.** Décrivez toutes les sorties possibles pour le programme, en sachant qu’il est séquentiellement cohérent.
- 3c.** Décrivez la machine à états du programme.

Exercice 4 (Le savant fou ¹).

En visite pédagogique dans un laboratoire, vous et le reste de votre groupe êtes capturés par un savant fou. Il fait l’annonce suivante :

« J’ai mis en place une pièce qui contient une lumière et un interrupteur qui la commande. Actuellement, la lumière est éteinte. Vous pouvez discuter ce soir d’une stratégie commune, mais à partir de demain vous serez isolés et ne pourrez plus communiquer. Chaque jour, je tirerai au hasard de manière uniforme l’un d’entre vous, qui passera la journée dans la pièce. À tout moment durant sa visite, il pourra déclarer : *Nous avons tous visité la pièce*. S’il a raison, vous êtes libres. Sinon, je vous reboote les cerveaux ! »

Hypothèses implicites :

- La lumière est éteinte au départ.
- L’interrupteur est binaire (allumé/éteint).
- La lumière est invisible depuis l’extérieur de la pièce, et il n’y a aucun autre moyen de communication entre les étudiants.

- 4a.** Énoncez précisément ce que doit garantir une stratégie réussie.
- 4b.** Proposez une stratégie et justifiez brièvement en quoi elle satisfait l’exigence précédente.
- 4c.** Reprenez la question précédente, sans l’hypothèse sur l’état initial de la lumière.
- 4d.** Reprenez les deux premières questions, sans l’hypothèse que le tirage est uniforme. À la place, on suppose que si personne ne déclare que tout le monde a visité la pièce, chaque étudiant est sélectionné une infinité de fois (pas nécessairement à intervalles bornés).

1. Inspiration : M. Herlihy & N. Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann (2008)